



UNIVERSITÀ DI PISA

Data Science and Business Informatics

Laboratory of Data Science

Decision Support System Per Una Federazione Tennistica

Cattari Simona e Trivelli Matteo

2024-2025

Contents

1	Assignment 1: Data understanding	2
1.1	Missing Values	2
2	Assignment 2: Data cleaning	3
2.1	Fact	3
2.2	Geography	3
2.3	Tourney	3
3	Assignment 3: Data schema	4
3.0.1	Data Preparation	5
4	Data uploading	5
4.0.1	Python	5
4.0.2	SSIS	6
4.1	Assignment 6	6
5	SSIS	6
5.1	Assignment 7	6
5.2	Assignment 8	7
6	SSAS	9
6.1	Query MDX	10
6.1.1	Assignment 9	10
6.1.2	Assignment 10	10
6.1.3	Assignment 11	10
7	Dashboard	11
7.0.1	Assignment 12	11
7.0.2	Assignment 13	12

Assignment 2: Data cleaning

All'interno di questa sezione vengono spiegate le singole operazioni effettuate sui vari dataset per gestire i missing values.

2.1 Fact

- **IOC e caratteristiche giocatori:** Le colonne `winner_ioc`, `loser_ioc`, `winner_hand` e `loser_hand` sono state completate mappando le informazioni note per ciascun giocatore tramite `winner_id` e `loser_id`, garantendo coerenza nei dati.
- **Ranking:** Per `winner_rank`, `winner_rank_points`, `loser_rank` e `loser_rank_points`, si è verificata la presenza di un valore nei match precedenti o successivi. In assenza, è stata utilizzata la media dei ranking nel torneo; se ancora non disponibile, è stato assegnato 0.
- **Età e altezza:** `winner_age` e `loser_age` sono stati imputati con la mediana del torneo o, in mancanza, con la mediana globale. Per `winner_ht` e `loser_ht`, si è preferito il valore noto del giocatore, altrimenti la mediana globale.
- **Score:** Se disponibile per lo stesso torneo, è stato riutilizzato; altrimenti, è stato assegnato "Unknown".
- **Draw size:** Dopo la conversione dei dati e la rimozione di "R", i valori mancanti sono stati riempiti con quelli già presenti nel torneo o stimati in base ai partecipanti unici (`winner_id` e `loser_id`).
- **Superficie di gioco:** Nel caso in cui `surface` fosse nota in altre partite dello stesso torneo, è stata mantenuta; in caso contrario, è stato assegnato "Unknown".

2.2 Geography

- **Aggiornamento *country_code* obsoleti:** I codici IOC obsoleti nel dataset `fact` sono stati sostituiti con quelli attuali utilizzando un dizionario di mappatura creato verificando le corrispondenze tra vecchi e nuovi codici tramite il sito <https://www.worlddata.info>. I valori mancanti sono stati impostati su UNK.
- **Recupero informazioni geografiche:** Per i codici IOC dei giocatori che non risultavano registrati all'interno della tabella `countries`, sono state effettuate chiamate API a `restcountries.com` per ottenere il nome del paese e il continente. Se i dati non erano disponibili, sono stati impostati valori di default (Unknown).

2.3 Tourney

Nel dataset `tourney`, dopo aver eliminato i duplicati, il processing dei dati si è concentrato principalmente sul trattamento della feature `tourney_timestamp`, in cui abbiamo calcolato la mediana dei valori positivi e sostituito i timestamp negativi col valore ottenuto. Infine, da

ciascun timestamp abbiamo estratto i rispettivi valori di *date*, *year*, *quarter*, *month*, *day*, *day of week*, *The Month* e *The Day*.

Assignment 3: Data schema

Il design scelto per il data warehouse si compone di 6 tabelle:

- **tournament**: contiene le informazioni relative ai tornei, inclusi nome, dimensione del tabellone e superficie di gioco.
- **match_fact**: raccoglie i dettagli di ogni partita disputata, con riferimenti al torneo, ai giocatori coinvolti e alle statistiche del match.
- **player**: conserva i dati anagrafici e fisici dei giocatori, compresa la loro nazionalità.
- **date**: memorizza le informazioni temporali relative allo svolgimento dei tornei e delle partite.
- **match_stats**: registra le statistiche di ogni incontro, incluse informazioni sul ranking e sulle prestazioni dei giocatori.
- **geography**: definisce la provenienza geografica dei giocatori, associando ciascuno a una nazione e a un continente.

All'interno della fact table **match_fact** sono presenti due chiavi esterne che puntano alla tabella **player**, **winner_fk** e **loser_fk**. Questa scelta di design si è basata sulla necessità di ottimizzare le query relative ai giocatori, permettendo un accesso diretto ai dati dei vincitori e degli sconfitti, riducendo anche il numero di join necessari.

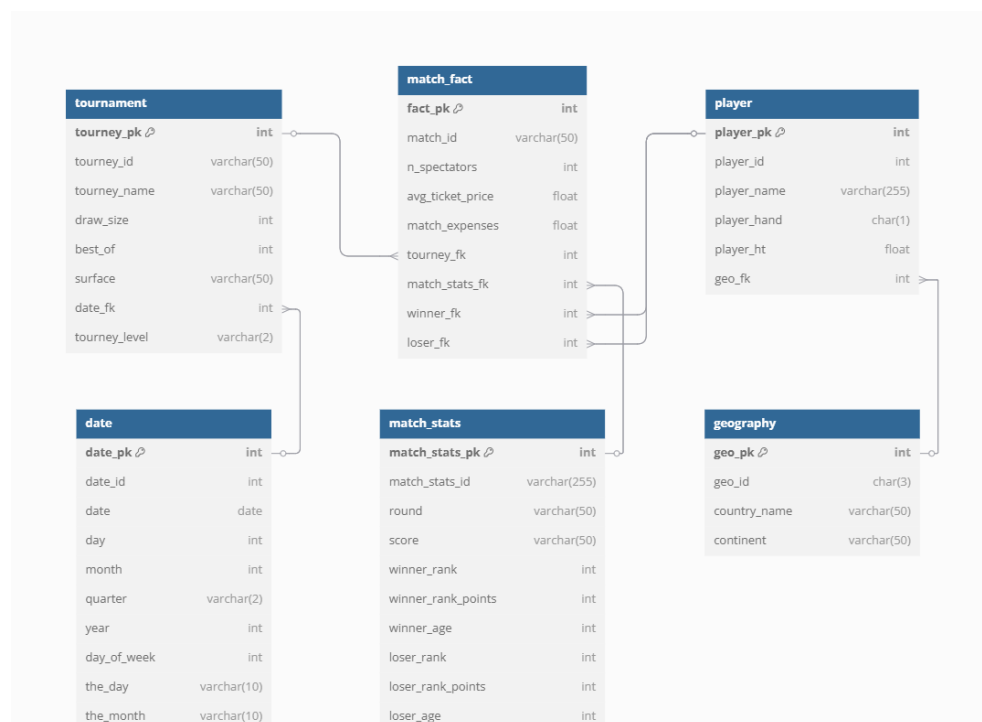


Figure 3.1: Schema del Datawarehouse

3.0.1 Data Preparation

Per la costruzione del data warehouse, i dati sono stati trasformati nelle tabelle presentate in 5.2 nel seguente modo:

- **match_fact**, **match_stats** e **player** sono stati estratti e strutturati a partire dalla tabella di origine **fact**, suddividendo le informazioni relative agli incontri, alle statistiche di gioco e ai dati relativi ai giocatori.
- **tournament** e **date** sono stati derivati dalla tabella **tourney**, separando i dettagli dei tornei dalle informazioni temporali.
- **geography** è stata costruita a partire dal file **countries.xml**, normalizzando i dati per associare ogni giocatore alla sua nazione e continente di appartenenza.

Questa fase di preparazione ha garantito un'organizzazione ottimale dei dati, riducendo ridondanze e migliorando le performance delle query analitiche.

Data uploading

Per effettuare il caricamento delle tabelle precedentemente create, sono state adottate due strategie per il caricamento di un sample del 20% dei dati mantenendo l'integrità referenziale al fine di determinare il metodo più veloce per eseguire il caricamento completo. Le due strategie comprendono l'uso di Python e l'uso di SSIS. Prima di eseguire queste operazioni, all'interno di SQL Server Management Studio sono state duplicate le tabelle e sono state rinominate con il suffisso '_SSIS' quelle destinate al caricamento tramite SSIS e con il suffisso '_SSIS_PY' quelle destinate al caricamento tramite python.

4.0.1 Python

Il processo è stato implementato sfruttando la libreria *pyodbc* per la connessione a SQL Server e *csv* per la gestione dei file di origine. Per migliori prestazioni è stata attivata l'opzione **fast_executemany** che consente di eseguire più inserimenti in un'unica operazione, migliorando prestazioni ed efficienza del processo. I dati sono contenuti in file CSV, il codice estrae un campione del 20% attraverso la funzione **sample_data**. Per estrarre il campione in modo coerente con le relazioni tra le tabelle, viene seguita una logica ben precisa:

- **Tournament:** Viene estratto un campione del 20% dei tornei e si salvano le chiavi primarie per collegarle ai dati successivi;
- **Match_fact:** Vengono selezionati solo i match che appartengono ai tornei selezionati in modo da mantenere la coerenza tra le due tabelle;
- **Players:** Si selezionano i giocatori che hanno partecipato a questi match;
- **Geography:** Ogni giocatore ha una posizione geografica associata, quindi vengono selezionate anche le informazioni geografiche relative ai giocatori;
- **Match_stats:** Le statistiche delle partite vengono filtrate per includere solo quelle relative ai match già selezionati;

- **Date:** Si selezionano le date dei tornei per mantenere la relazione con gli eventi.

Questa logica ha permesso di mantenere l'integrità referenziale e quindi la coerenza dei dati tra le tabelle. Successivamente, tramite la funzione `insert_data` ogni tabella viene popolata rispettando l'ordine logico delle dipendenze tra le tabelle.

4.0.2 SSIS

Anche in questo caso si segue una logica gerarchica per evitare errori di chiavi durante l'inserimento dei dati. Il primo step consiste nel popolamento delle tabelle **Geography**, **Date** e **Match_stats** in quanto non hanno dipendenze da altre tabelle e quindi l'inserimento avviene in modo diretto. I dati vengono estratti tramite **File flat source** dove si trovano i file csv delle tabelle corrispondenti e successivamente viene selezionato un campionamento del 20% tramite il nodo **Row Sampling** e infine vengono caricati nelle rispettive tabelle tramite **OLE DB Destination**. Il secondo step coinvolge il popolamento delle tabelle **Players** e **Tournament** che contengono riferimenti ad altre tabelle, rispettivamente a **Geography** e **Date**. Per queste due tabelle, dopo aver estratto i file con **File flat source** e aver selezionato il 20% dei dati con **Row Sampling**, è stato effettuato un **Lookup** per associare correttamente le chiavi esterne per mantenere le relazioni. Come ultimo step è stata popolata la fact table (**Match_fact**) che contiene i riferimenti a tutte le tabelle sopra citate. Anche in questo caso sono stati estratti i dati dal file csv tramite **File flat source**, è stato fatto il campionamento del 20% con il nodo **Row Sampling** e infine sono stati eseguiti una serie di **Lookup** per garantire l'integrità dei dati, in particolare:

- *Lookup* su **Tournament** per recuperare *tourney_fk*;
- Due *Lookup* sulla tabella **Players** per ottenere le chiavi *winner_fk* e *loser_fk*;
- *Lookup* sulla tabella **Match_stats** per ottenere la chiave *match_stats_fk*

Dopo queste operazioni, l'output è stato caricato tramite **OLE DB Destination** alla tabella corrispondente.

4.1 Assignment 6

In base alle performance dei due approcci descritti in precedenza, l'approccio basato su Python con la libreria *pyodbc* e l'opzione *fast_executemany* ha mostrato tempi di esecuzione più rapidi rispetto al workflow SSIS. Per questo motivo è stato deciso di usare questo approccio per il caricamento completo dei dati nel database.

SSIS

In questa sezione verranno mostrati e discussi i Workflow SSIS sviluppati.

5.1 Assignment 7

For any given player, his or her nemesis is the player against whom he or she lost most matches. For each year, list every player with the respective nemesis and the number of matches lost

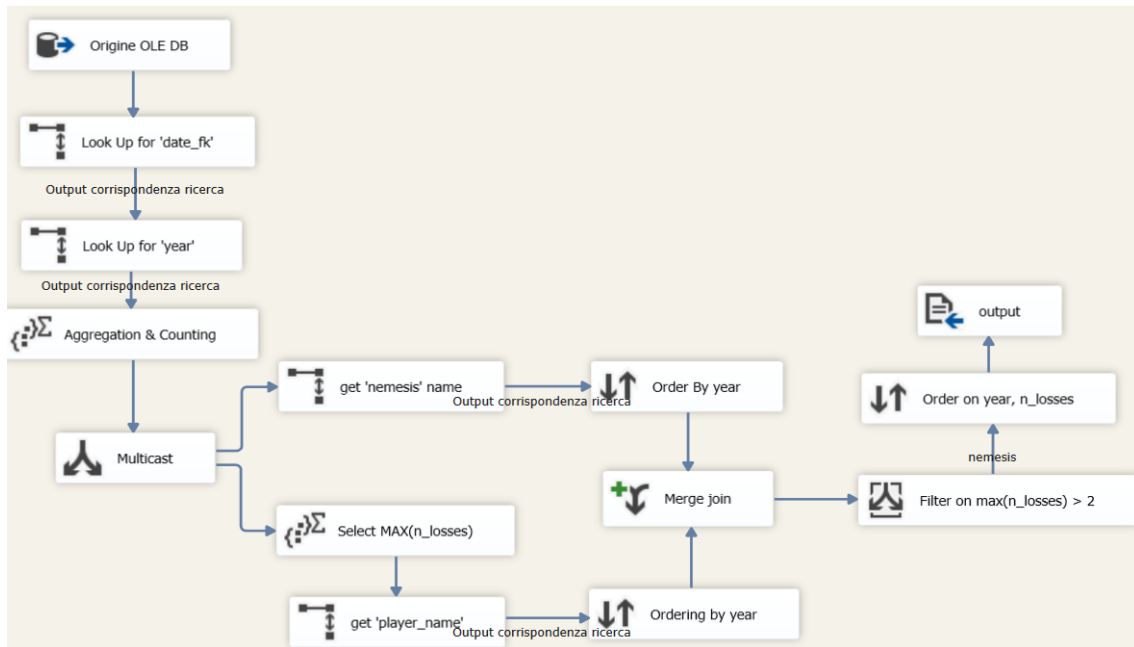


Figure 5.1: SSIS Workflow

Dopo aver estratto `winner_fk`, `loser_fk` e `tourney_fk` dalla `fact table`, mediante due operazioni di *LookUp* si è recuperato `year` dalla tabella `date`. Successivamente, si è eseguita un'aggregazione sulle chiavi esterne relative ai giocatori e all'anno, con una funzione `COUNT()` su `loser_fk`, ottenendo così il numero di sconfitte che un giocatore ha subito contro lo stesso avversario. Qui, `loser_fk` è stato rinominato in `player_id` e `winner_fk` in `nemesys`. I dati ottenuti sono stati quindi duplicati in due tabelle distinte tramite un'operazione di *Multicast*: una utilizzata per trovare il massimo numero di sconfitte per ciascun giocatore in un dato anno, effettuando un'aggregazione su `player_id` e `year`, con la funzione `MAX()` su `n_losses`. Dopo aver ordinato entrambe le tabelle sugli stessi attributi, si è eseguito un *Merge Join* su `player_id` e `year`, permettendo così di confrontare, nel nodo successivo, il numero massimo di sconfitte di un giocatore in un dato anno con il numero di sconfitte contro i vari avversari. Infine, attraverso un *conditional split*, confrontando `n_losses` e `max_losses`, è stato possibile individuare il `nemesys` di ciascun giocatore per ciascun anno.

5.2 Assignment 8

*A match is labeled with age-outlier if the difference in years between the winner and the loser is more than $\text{avg}(\text{agedifference}) * 1.5$ w.r.t. all the matches in the same tournament. For each year, list the player that participated in the most age outlier matches.*

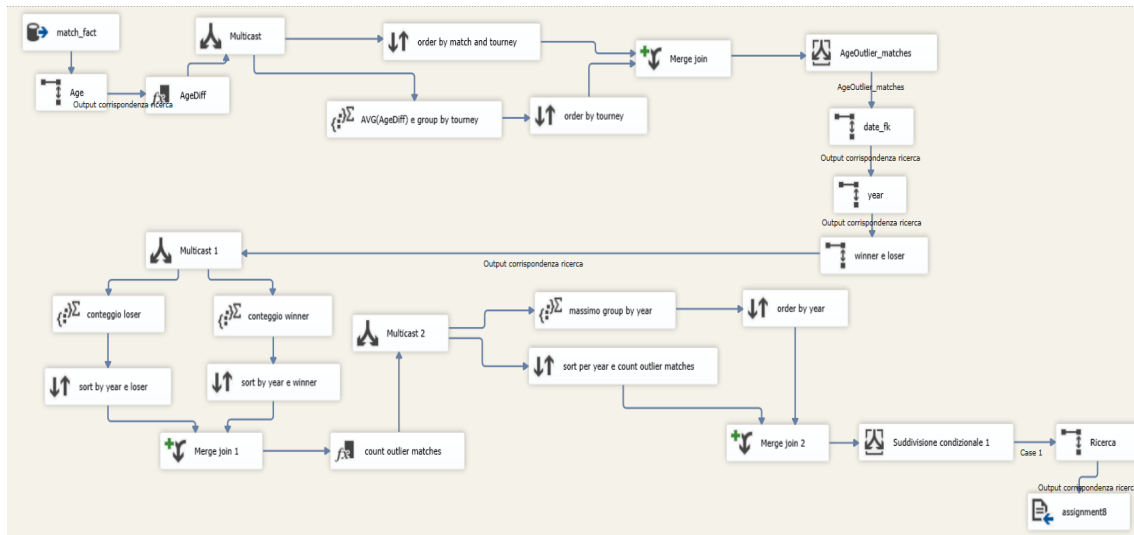


Figure 5.2: SSIS Workflow

FASE 1: Il flusso dati inizia con l'estrazione dei match dalla fact table **match_fact**. Successivamente viene effettuato un *Lookup* sulla tabella **match_stats** per recuperare le età dei giocatori vincitori e sconfitti. Questo passaggio è stato fondamentale per calcolare, tramite *Colonna Derivata*, il campo **AgeDiff** che rappresenta la differenza assoluta tra le età del vincitore e del perdente. Successivamente tramite *Multicast* il dataset viene duplicato creando due percorsi distinti che verranno successivamente uniti tramite un *Merge Join* sulla chiave del torneo. Questa operazione permette di associare a ogni match la media della differenza di età nel torneo corrispondente:

- Nel primo percorso viene calcolata la media della differenza di età ($AVG(AgeDiff)$) per ogni torneo, raggruppando per torneo.
- Nel secondo percorso i dati vengono ordinati per match e torneo per garantire una corretta unione con l'altro flusso.

Infine, una *Suddivisione Condizionale* viene usata per filtrare e conservare solo i match che soddisfano la condizione per considerare un match age-outlier ($AgeDiff > (AVG(AgeDiff) * 1.5)$).

FASE 2: In questa fase, tramite *Lookup*, vengono estratte informazioni relative alle date, in particolare l'anno del match, oltre alle chiavi dei giocatori coinvolti. Per poter contare separatamente le partecipazioni ai match age-outlier per vincitori e sconfitti, il dataset viene nuovamente duplicato tramite un *Multicast*. Successivamente, i due output vengono uniti con un *Merge Join* sulla coppia (**player_id**, **year**), ottenendo così il totale delle partecipazioni per ogni giocatore:

- Nel primo flusso, viene contato il numero di volte in cui un vincitore ha partecipato a un match age-outlier, raggruppando i risultati per **year** e **winner_fk** (che diventa **player_id**).
- Nel secondo flusso, viene eseguita la stessa operazione per il giocatore sconfitto.

FASE 3: A questo punto, vengono individuati i giocatori che hanno partecipato a più match age-outlier per anno. Utilizzando una *Colonna Derivata*, si calcola il numero totale

di partecipazioni per ciascun giocatore, sommando le partecipazioni come vincitore e come sconfitto. Successivamente, un nuovo *Multicast* consente di gestire due operazioni parallele:

- Nel primo flusso, i dati vengono aggregati per anno per determinare il massimo numero di match age-outlier disputati da un singolo giocatore.
- Nel secondo flusso, viene eseguito un ordinamento per anno e numero di match age-outlier.

I risultati dei due percorsi vengono poi combinati tramite un *Merge Join*, ottenendo così, per ogni anno, il giocatore con il maggior numero di match age-outlier.

FASE 4: Nell'ultima fase, vengono recuperati i nomi dei giocatori. Una *Suddivisione Condizionale* filtra i dati selezionando solo le righe in cui il numero di match age-outlier è pari al valore massimo, in modo da includere esclusivamente i giocatori con il maggior numero di match age-outlier per ogni anno. Infine, tramite un *Lookup* sulla tabella **players**, vengono estratti i nomi dei giocatori e l'output finale viene salvato in un file flat.

SSAS

Per la creazione del datacube, la scelta è stata quella di mantenere tutti gli attributi di ciascuna dimensione oltre a quelli necessari per rispondere alle business questions. Questo permette, in futuro, di garantire maggiore flessibilità nell'analisi dei dati, facilitando l'aggiunta di nuove misure o la definizione di gerarchie più dettagliate senza dover ristrutturare il modello. Le dimensioni sono state gestite come segue:

- **Date:** *date_id* è stato usato come chiave e *date* come elemento minimo. È stata definita una singola gerarchia di interesse: *DayMonthYear*, la cui granularità va dall'anno alla data. Successivamente, per gestire l'ordinamento dei mesi e dei giorni della settimana, di default ordinati alfabeticamente, sono state definite le seguenti dipendenze funzionali:

- *The Month* $\rightarrow$ *Month* (es. *January* $\rightarrow$ 1)
- *The Day* $\rightarrow$ *Day Of Week* (es. *Monday* $\rightarrow$ 1)

Infine, tali valori numerici sono stati utilizzati per l'ordinamento nelle *properties* di entrambi gli attributi.

- **Geography:** *geo_pk* è stato utilizzato come chiave e *geo_id* elemento minimo. Anche in questo caso è stata definita una gerarchia d'interesse: *Geo*, che va da *Continent* a *IOC code*. Infine si è stabilita la seguente dipendenza funzionale:

- *IOC code* $\rightarrow$ *Country Name* $\rightarrow$ *Continent*.

- **Match Fact:** per facilitare la risposta alle business question è stata definita la misura *Profit* come: $(avg_ticket_price * n_spectators) - match_expenses$.
- **Match Stats, Tournament, Player:** sono state configurate senza la necessità di definire gerarchie. Di conseguenza, sono state specificate solo le chiavi e gli attributi.

Dopo la creazione del cubo le misure sono le seguenti: *n Spectators*, *Match Expenses*, *n Matches* e *Profit*.

6.1 Query MDX

6.1.1 Assignment 9

Show the player that lost the most matches for each continent.

```
WITH MEMBER n_losses
AS [Measures].[n Matches]

SELECT n_losses ON COLUMNS,
([Loser].[Nationality].[Continent], GENERATE ([Loser].[Nationality].[Continent],
TOPCOUNT([Loser].[Player Name].[Player Name], 1, [Measures].[n Matches]))) ON ROWS
FROM [Matches];
```

Figure 6.1: MDX Query Assignment 9

Per individuare il giocatore che ha perso il numero maggiore di partite per ciascun continente è stata utilizzata la funzione `TOPCOUNT()` per ordinare i giocatori in base al numero di sconfitte, selezionando solo il primo giocatore della lista. La funzione `GENERATE()` applica questa selezione a ciascun continente, restituendo per ciascuno il giocatore più sconfitto.

6.1.2 Assignment 10

For each tournament, show the number of players.

```
WITH
MEMBER [Measures].[Player Count] AS
    DistinctCount(
        Union(
            [Winner].[Player Id].[Player Id],
            [Loser].[Loser Id].[Loser Id]
        )
    )

SELECT
    [Measures].[Player Count] ON COLUMNS,
    NON EMPTY [tournament].[Tourney Name].[Tourney Name] ON ROWS
FROM [Matches]
```

Figure 6.2: MDX Query Assignment 10

Per ottenere il numero univoco di giocatori per ciascun torneo, è stato creato un membro calcolato (**Player Count**) che utilizza la funzione `DistinctCount` per contare in modo univoco gli elementi di un insieme. Poiché un giocatore può apparire sia come vincitore che come sconfitto nelle partite, è stata utilizzata la funzione `Union` per combinare i due insiemi assicurando così che ogni giocatore venga conteggiato una sola volta per torneo. Il membro calcolato **Player Count** è posizionato sulle colonne, restituendo il numero di giocatori unici per ogni torneo. Sulle righe, invece, viene posizionata la gerarchia **Tourney Name** della dimensione **tournament**, in modo da visualizzare il conteggio suddiviso per torneo. Inoltre, la clausola `NON EMPTY` viene utilizzata per escludere eventuali tornei senza giocatori associati, ottimizzando la visualizzazione dei risultati.

6.1.3 Assignment 11

For each quarter, calculate the percentage increase or decrease in profit for each tournament compared to the profit in the same tournament during the corresponding quarter of the previous year.

```

WITH MEMBER prev_year_profit AS
IIF(
    IEMPTY(
        (PARALLELPERIOD([Tournament].[DayMonthYear].[Year],
            1,
            [Tournament].[DayMonthYear].CURRENTMEMBER)
        , [Measures].[Profit])
    ),
    0,
    (PARALLELPERIOD([Tournament].[DayMonthYear].[Year],
        1,
        [Tournament].[DayMonthYear].CURRENTMEMBER)
    , [Measures].[Profit])
)

MEMBER pct AS
IIF(
    IEMPTY(prev_year_profit) OR prev_year_profit = 0,
    0,
    (([Measures].[Profit] - prev_year_profit) / (prev_year_profit))
)
, FORMAT_STRING = 'Percent'

SELECT {[Measures].[Profit], prev_year_profit, pct} ON COLUMNS,
    ([Tournament].[Tourney Name].[Tourney Name],
    [Tournament].[Year].[Year],
    [Tournament].[DayMonthYear].[Quarter]) ON ROWS
FROM [Matches];

```

Figure 6.3: MDX Query Assignment 11

Per calcolare la variazione percentuale dei profitti è stata utilizzata la funzione `PARALLELPERIOD()` per ottenere il valore del profitto dell'anno precedente per ogni torneo. Il valore viene poi confrontato con il profitto attuale per calcolare la variazione percentuale. La funzione `IIF()` gestisce i casi in cui il profitto dell'anno precedente sia nullo o pari a zero, evitando errori di calcolo. Infine, il risultato viene formattato come percentuale e visualizzato insieme ai profitti correnti e ai profitti dell'anno precedente per ciascun torneo e trimestre.

Dashboard

7.0.1 Assignment 12

Create a dashboard that shows the geographical distribution of winner rank points and loser rank points.

All'interno di questa prima dashboard vengono rappresentate le aree del mondo con la maggiore presenza di giocatori competitivi, evidenziando la distribuzione globale dei loro punteggi nel ranking. Per garantire una visualizzazione più chiara e intuitiva, i punteggi sono stati suddivisi in intervalli attraverso la tecnica dell'equal frequency binning, che permette di definire classi bilanciate in base alla frequenza dei valori. Sulla base di questa suddivisione, le variabili `winner_bin` e `loser_bin` sono state aggiunte al cubo come calcoli denominati, consentendo un'analisi più strutturata e dettagliata della distribuzione dei giocatori. Questo approccio permette di rappresentare in maniera più chiara la distribuzione dei punti, evitando che la visualizzazione sia dominata da un numero ristretto di giocatori con punteggi estremamente elevati. In questo modo è possibile individuare non solo le nazioni con i giocatori di vertice, ma anche quelle con una presenza significativa di atleti di livello medio-alto, offrendo un quadro più completo della competitività a livello globale.

Mappa dei Punti in Classifica ATP/WTA: Perdenti vs Vincitori

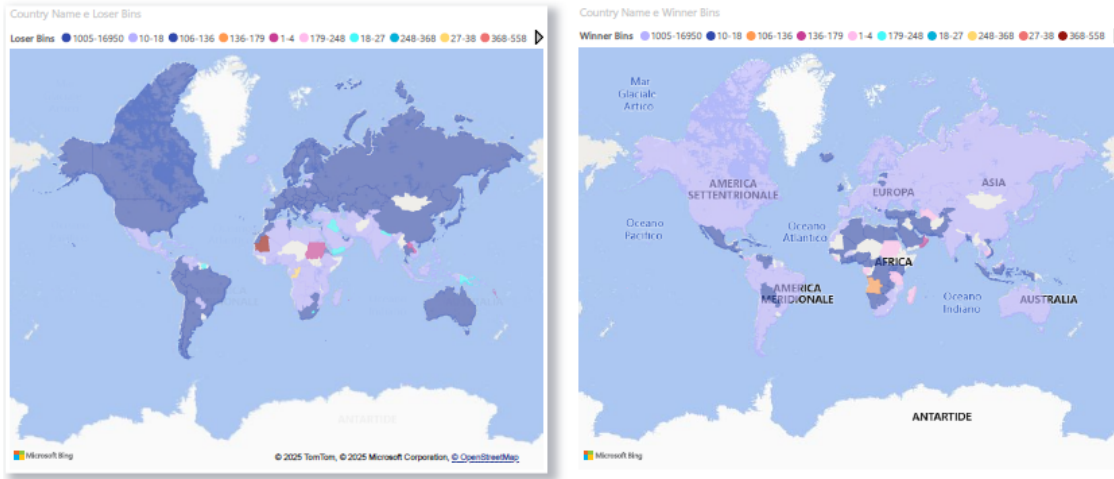


Figure 7.1: Dashboard Assignment 12

7.0.2 Assignment 13

Create a plot/dashboard that you deem interesting, including the data available in your cube, focusing on the financial informations.

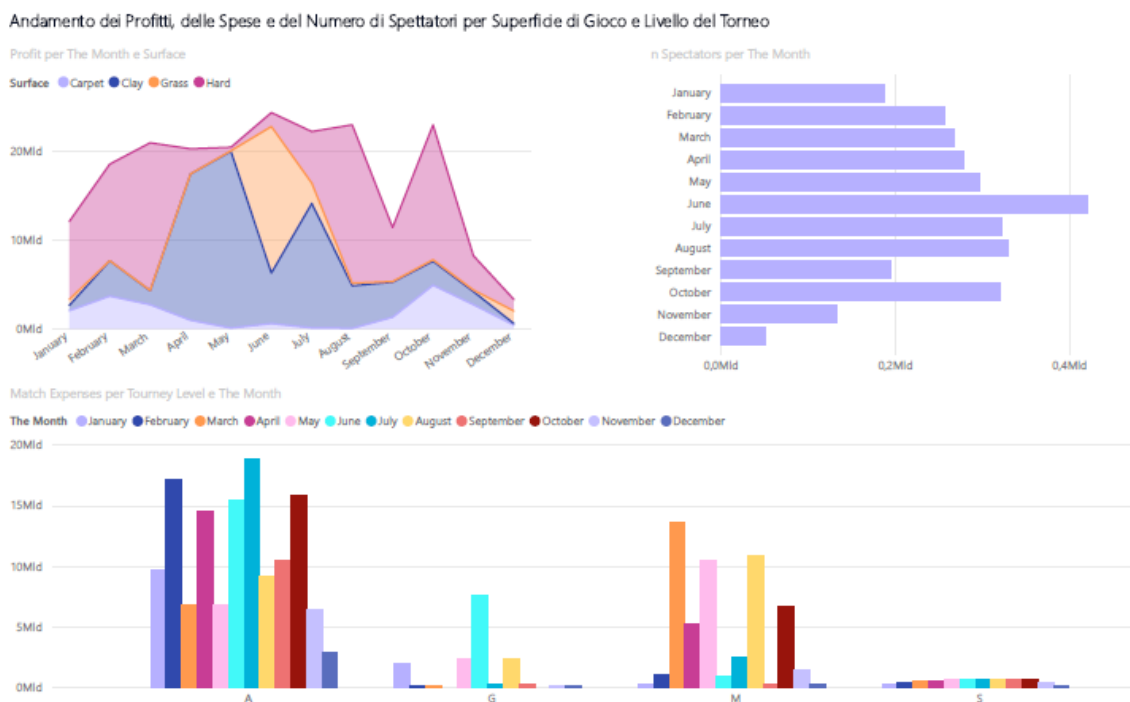


Figure 7.2: Dashboard Assignment 13

La dashboard in questione si focalizza sull'andamento dei profitti per superficie di gioco, sulla distribuzione del numero di spettatori nel corso dell'anno e sulle spese per livello di torneo. Il primo grafico mostra come i profitti varino a seconda della superficie, con picchi evidenti nei mesi in cui probabilmente si disputano i tornei più prestigiosi. Questo trend evidenzia l'impatto del calendario degli eventi tennistici sui ricavi complessivi. Il secondo grafico, dedicato agli spettatori, conferma un'affluenza molto elevata nei mesi estivi, mentre nei mesi invernali il numero di spettatori tende generalmente ad essere più basso. Infine, il terzo grafico analizza

le spese sostenute nei tornei di diverso livello, evidenziando come tornei di livello molto alto comportino costi nettamente superiori rispetto agli eventi di categoria inferiore; in questo caso si è deciso di mostrare solamente le categorie di torneo per cui si aveva un numero abbastanza significativo di dati.